

Algorithm:

Main Program

1. Start.
2. Display the menu:
 - Left distributive property
 - Right distributive property
 - Exit
3. Read the user's choice.
4. Use switch:
 - If choice is 1, call leftDistributive().
 - If choice is 2, call rightDistributive().
 - If choice is 3, exit.
 - Otherwise, display an invalid choice message.
5. Repeat until the user selects Exit.
6. Stop.

Left Distributive Property

To verify:

$$\forall [A(B+C)=(AB)+(AC) \ \forall]$$

1. Read positive values of m, n, and p.
2. Declare:
 - $A[m][n]$
 - $B[n][p]$
 - $C[n][p]$
3. Read all three matrices.
4. Calculate $B + C$.
5. Calculate $A \times (B + C)$ and store it as LHS.
6. Calculate $A \times B$.

7. Calculate $A \times C$.
8. Add $AB + AC$ and store it as RHS.
9. Display LHS and RHS.
10. Compare corresponding elements.
11. If all elements are equal, the left distributive property is verified.

Right Distributive Property

To verify:

$$\backslash [(A+B)C = (AC) + (BC) \backslash]$$

1. Read positive values of m, n, and p.
2. Declare:
 - $A[m][n]$
 - $B[m][n]$
 - $C[n][p]$
3. Read all three matrices.
4. Calculate $A + B$.
5. Calculate $(A + B) \times C$ and store it as LHS.
6. Calculate $A \times C$.
7. Calculate $B \times C$.
8. Add $AC + BC$ and store it as RHS.
9. Display LHS and RHS.
10. Compare corresponding elements.
11. If all elements are equal, the right distributive property is verified.

C Program

```
#include <stdio.h>
```

```
/* Function prototypes */
```

```
void leftDistributive(void);
```

```
void rightDistributive(void);
```

```
void readMatrix(int r, int c, int a[r][c]);
```

```
void printMatrix(int r, int c, int a[r][c]);
```

```
void addMatrix(int r, int c,  
               int a[r][c], int b[r][c], int result[r][c]);
```

```
void multiplyMatrix(int r1, int c1, int c2,  
                   int a[r1][c1], int b[c1][c2],  
                   int result[r1][c2]);
```

```
int compareMatrix(int r, int c,  
                  int a[r][c], int b[r][c]);
```

```
int main()  
{  
    int choice;  
  
    do  
    {  
        printf("\n----- MENU ----- \n");  
        printf("1. Verify Left Distributive Property\n");  
        printf("2. Verify Right Distributive Property\n");  
        printf("3. Exit\n");  
  
        printf("Enter your choice: ");  
        scanf("%d", &choice);
```

```
switch (choice)
{
    case 1:
        leftDistributive();
        break;

    case 2:
        rightDistributive();
        break;

    case 3:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1, 2 or 3.\n");
}

} while (choice != 3);

return 0;
}
```

```
/* Function to read a matrix */
void readMatrix(int r, int c, int a[r][c])
{
    int i, j;
```

```
for (i = 0; i < r; i++)  
{  
    for (j = 0; j < c; j++)  
    {  
        scanf("%d", &a[i][j]);  
    }  
}  
}
```

```
/* Function to display a matrix */  
void printMatrix(int r, int c, int a[r][c])  
{  
    int i, j;  
  
    for (i = 0; i < r; i++)  
    {  
        for (j = 0; j < c; j++)  
        {  
            printf("%d\t", a[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
/* Function to add two matrices */
```

```

void addMatrix(int r, int c,
               int a[r][c], int b[r][c], int result[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}

```

/* Function to multiply two matrices */

```

void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1], int b[c1][c2],
                   int result[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

```

```

        for (k = 0; k < c1; k++)
        {
            result[i][j] += a[i][k] * b[k][j];
        }
    }
}
}

```

/* Function to compare two matrices */

```

int compareMatrix(int r, int c,
                  int a[r][c], int b[r][c])

```

```

{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
            {
                return 0;
            }
        }
    }
}

```

```

    return 1;
}

```

```

/* Function to verify  $A(B+C) = AB + AC$  */
void leftDistributive(void)
{
    int m, n, p;

    printf("\n--- Left Distributive Property ---\n");
    printf("A(B+C) = (AB) + (AC)\n");

    do
    {
        printf("Enter m, n and p: ");
        scanf("%d %d %d", &m, &n, &p);

        if (m <= 0 || n <= 0 || p <= 0)
        {
            printf("Dimensions must be positive. Enter again.\n");
        }

    } while (m <= 0 || n <= 0 || p <= 0);

    /* Variable-length arrays */
    int A[m][n];
    int B[n][p];
    int C[n][p];

    int BC[n][p];

```

```
int AB[m][p];
int AC[m][p];

int LHS[m][p];
int RHS[m][p];

printf("\nEnter elements of A (%d x %d):\n", m, n);
readMatrix(m, n, A);

printf("\nEnter elements of B (%d x %d):\n", n, p);
readMatrix(n, p, B);

printf("\nEnter elements of C (%d x %d):\n", n, p);
readMatrix(n, p, C);

/* Calculate B + C */
addMatrix(n, p, B, C, BC);

/* LHS = A(B+C) */
multiplyMatrix(m, n, p, A, BC, LHS);

/* Calculate AB and AC */
multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, n, p, A, C, AC);

/* RHS = AB + AC */
addMatrix(m, p, AB, AC, RHS);
```

```
printf("\nLeft-hand side A(B+C):\n");
```

```
printMatrix(m, p, LHS);
```

```
printf("\nRight-hand side (AB)+(AC):\n");
```

```
printMatrix(m, p, RHS);
```

```
if (compareMatrix(m, p, LHS, RHS))
```

```
{
```

```
    printf("\nLeft distributive property is verified.\n");
```

```
}
```

```
else
```

```
{
```

```
    printf("\nLeft distributive property is not verified.\n");
```

```
}
```

```
}
```

```
/* Function to verify  $(A+B)C = AC + BC$  */
```

```
void rightDistributive(void)
```

```
{
```

```
    int m, n, p;
```

```
    printf("\n--- Right Distributive Property ---\n");
```

```
    printf("(A+B)C = (AC) + (BC)\n");
```

```
do
```

```
{
```

```
    printf("Enter m, n and p: ");
```

```

scanf("%d %d %d", &m, &n, &p);

if (m <= 0 || n <= 0 || p <= 0)
{
    printf("Dimensions must be positive. Enter again.\n");
}

} while (m <= 0 || n <= 0 || p <= 0);

/* Variable-length arrays */
int A[m][n];
int B[m][n];
int C[n][p];

int AB[m][n];
int AC[m][p];
int BC[m][p];

int LHS[m][p];
int RHS[m][p];

printf("\nEnter elements of A (%d x %d):\n", m, n);
readMatrix(m, n, A);

printf("\nEnter elements of B (%d x %d):\n", m, n);
readMatrix(m, n, B);

printf("\nEnter elements of C (%d x %d):\n", n, p);

```

```
readMatrix(n, p, C);

/* Calculate A + B */
addMatrix(m, n, A, B, AB);

/* LHS = (A+B)C */
multiplyMatrix(m, n, p, AB, C, LHS);

/* Calculate AC and BC */
multiplyMatrix(m, n, p, A, C, AC);
multiplyMatrix(m, n, p, B, C, BC);

/* RHS = AC + BC */
addMatrix(m, p, AC, BC, RHS);

printf("\nLeft-hand side (A+B)C:\n");
printMatrix(m, p, LHS);

printf("\nRight-hand side (AC)+(BC):\n");
printMatrix(m, p, RHS);

if (compareMatrix(m, p, LHS, RHS))
{
    printf("\nRight distributive property is verified.\n");
}
else
{
    printf("\nRight distributive property is not verified.\n");
}
```

```
}  
  
}
```

TEST CASE TABLE:

TEST CASE	CHOICE	m n p	PURPOSE	EPECTED RESULT
1	1	2 2 2	Valid matrices	LHS = RHS; left distributive property verified
2	2	2 2 2	Valid matrices	LHS = RHS; right distributive property verified
3	1	1 1 1	Boundary case	Property verified
4	2	1 1 1	Boundary case	Property verified
5	1	0 2 2	Invalid dimension	Error; dimensions requested again
6	2	2 -1 2	Invalid dimension	Error; dimensions requested again
7	4	--	Invalid menu choice	Invalid choice!
8	3	--	Exit	Program terminates